Informe de tests - Intelligence System

Entrega final. Fecha de la corrida: 2026-10-22. Repositorio: https://github.com/osangale1/clases (rama `entregable-final`).

1. Resumen

Verificacion	Resultado
Suite de tests (pytest)	**61 de 61 pasan** (incluye 2 de integracion contra Redis Stack real)
Cobertura de `app/`	**93 %** (527 sentencias, 37 sin cubrir)
Revision de codigo con OpenCode (`gemini-3.5-flash-lite`)	Ver `REVISION_OPENCODE.md` : sin problemas de severidad media/alta
Corrida en una copia limpia, sin Redis	59 pasan, 2 se saltean solos (los de integracion)
Lint (`ruff check` , `ruff format --check`)	Sin errores
`docker-compose.yml`	Valido (servicios `redis` , `ingest` , `phoenix` , `app`)
Trazas en Arize Phoenix	7 pruebas trazadas (LLM con guion) + corrida real con Gemini
Las 7 pruebas de `scripts/pruebas.py` con Gemini real (tier gratuito)	**7 de 7 OK** (6 `DONE` + 1 rechazo 422 esperado), 54 llamadas reales al LLM trazadas en Phoenix

2. Como reproducirlo

```
 git clone https://github.com/osangale1/clases && cd clases
./run_tests.sh                                  # 59 pasan + 2 skipped (no necesita keys ni Docker)

docker run -d -p 6379:6379 redis/redis-stack-server
TEST_REDIS_URL=redis://localhost:6379 ./run_tests.sh   # 61 pasan
```

`run_tests.sh` crea el entorno virtual (Python 3.12), instala `requirements-dev.txt` , corre pytest y deja el resultado en `reports/test_report.txt` y `reports/junit.xml` .

3. Entorno

- Python 3.12.13, macOS (arm64).
- pytest 9.1, pytest-asyncio 1.4 (`asyncio_mode = auto`), fakeredis 2.40, pytest-cov.
- Redis Stack (`redis/redis-stack-server`) y Arize Phoenix (`arizephoenix/phoenix`) en Docker.

4. Que cubre cada archivo de tests

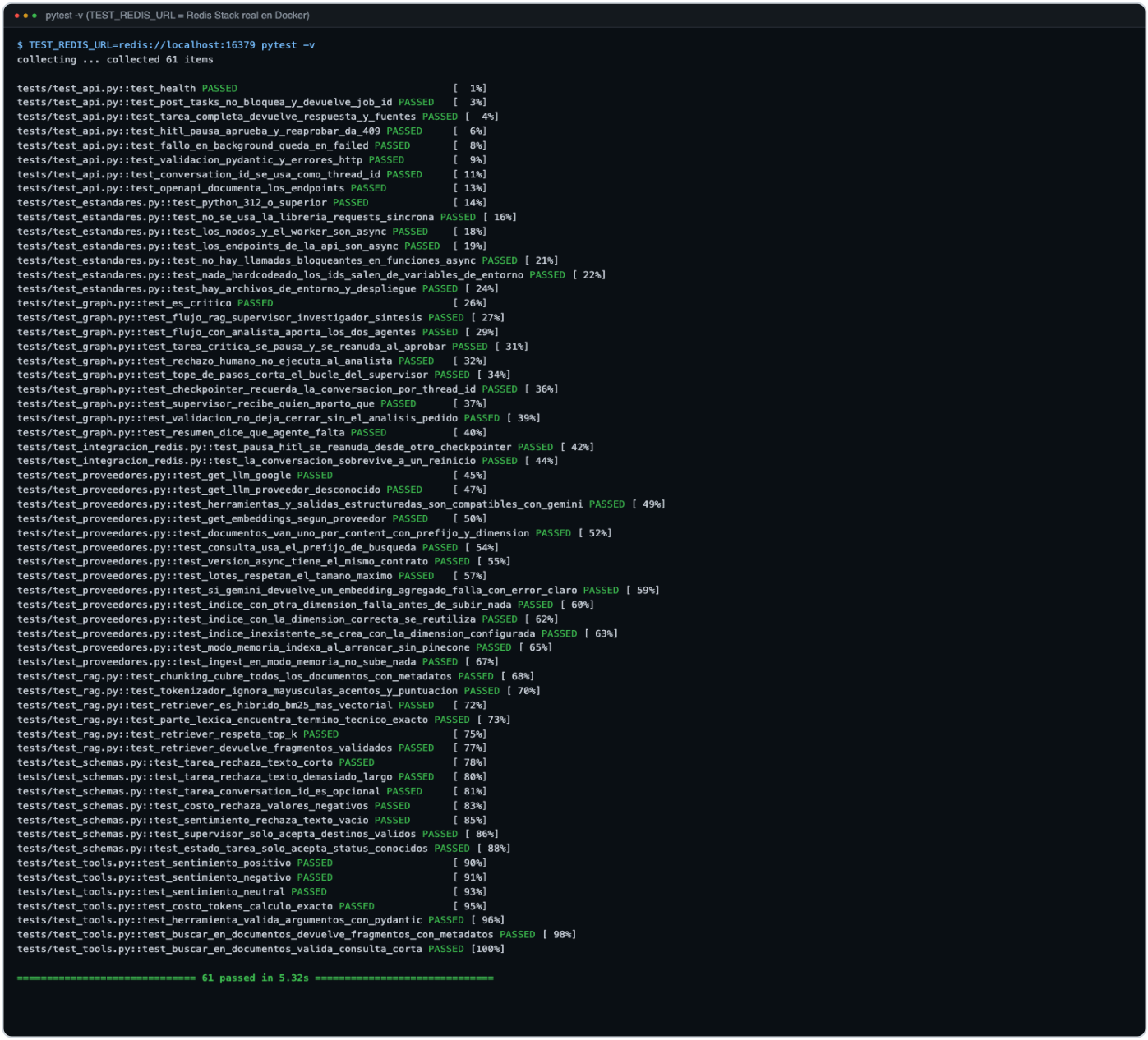
Archivo	Tests	Que verifica
`tests/test_api.py`	8	API real levantada con uvicorn: `POST /tasks` devuelve 202 en menos de 0.5 s sin esperar la tarea, flujo completo con respuesta y fuentes,

Archivo	Tests	Que verifica
		pausa humana + aprobacion + 409 al reaprobar, fallo en background queda en `FAILED`, validacion 422/404, `conversation_id` como `thread_id`, OpenAPI
`tests/test_graph.py`	10	Grafo LangGraph: supervisor -> investigador -> sintesis, flujo con analista, pausa y reanudacion (HITL), rechazo humano no ejecuta al analista, tope de pasos, memoria por `thread_id`, el supervisor recibe quien aporto que, validacion que no deja cerrar sin el analisis pedido
`tests/test_integracion_redis.py`	2	Contra Redis Stack real: una pausa HITL se retoma desde otra conexion del checkpointer (simula reinicio) y la conversacion sobrevive al reinicio
`tests/test_proveedores.py`	14	Gemini: seleccion de proveedor, esquemas de herramientas y salidas estructuradas compatibles, embeddings un `Content` por texto con prefijos y dimension, lotes, error si vuelve un embedding agregado, chequeo de dimension del indice de Pinecone, modo `VECTOR_STORE=memory`
`tests/test_rag.py`	6	Chunking con metadatos (fuente, pagina, categoria), tokenizador BM25, retriever hibrido BM25 + vectorial, `top_k`
`tests/test_tools.py`	7	Herramientas de los agentes: sentimiento, costo de tokens, busqueda RAG con metadatos, validacion Pydantic de argumentos
`tests/test_schemas.py`	7	Modelos Pydantic de la API y del supervisor: rechazan datos invalidos
`tests/test_estandares.py`	7	Python >= 3.12, sin `requests` sincrono, nodos/worker/endpoints `async`, sin llamadas bloqueantes dentro de `async def`, nada hardcodeado, archivos de entorno y despliegue presentes

Un control extra corre en **todos** los tests: ninguna llamada al LLM puede terminar en un mensaje del asistente (los modelos Claude nuevos lo rechazan con 400; fue un bug real de entregas anteriores).

5. Capturas

5.1 Suite completa con Redis Stack real (61 de 61)



5.2 Cobertura

pytest --cov=app

```
$ pytest --cov=app --cov-report=term-missing:skip-covered
============================= tests coverage ==============================
_____________ coverage: platform darwin, python 3.12.13-final-0 _____________

Name                    Stmts   Miss  Cover   Missing
-----------------------------------------------------
app/graph.py              105      3    97%   45, 187-188
app/llm.py                 12      4    67%   11-13, 16-18
app/observability.py       16      6    62%   18-27, 29
app/rag/embeddings.py      59      2    97%   29-30
app/rag/ingest.py          45     17    62%   15-18, 42, 47-49, 57-72, 76
app/rag/retriever.py       17      1    94%   19
app/rag/service.py         28      2    93%   25, 52
app/worker.py              36      2    94%   85-86
-----------------------------------------------------
TOTAL                     527     37    93%

12 files skipped due to complete coverage.
Coverage JSON written to file docs/coverage.json
```

Lo que queda sin cubrir es codigo que depende de servicios externos con cuenta: la creacion real del indice en Pinecone (`ingest.py`), el registro del colector de Phoenix (`observability.py`) y las ramas de OpenAI/Anthropic en `llm.py` (el proveedor por defecto, Gemini, si esta cubierto).

5.3 Copia limpia del repo, sin Redis ni API keys

./run_tests.sh en una copia limpia del repo (sin Redis)

```
$ git clone https://github.com/osangale1/clases && cd clases && ./run_tests.sh
tests/test_api.py::test_health PASSED                                    [  1%]
tests/test_api.py::test_post_tasks_no_bloquea_y_devuelve_job_id PASSED   [  3%]
tests/test_api.py::test_tarea_completa_devuelve_respuesta_y_fuentes PASSED [  4%]
tests/test_api.py::test_hitl_pausa_aprueba_y_reaprobar_da_409 PASSED     [  6%]
tests/test_api.py::test_fallo_en_background_queda_en_failed PASSED       [  8%]
tests/test_api.py::test_validacion_pydantic_y_errores_http PASSED        [  9%]
tests/test_api.py::test_conversation_id_se_usa_como_thread_id PASSED     [ 11%]
tests/test_api.py::test_openapi_documenta_los_endpoints PASSED           [ 13%]
tests/test_graph.py::test_es_critico PASSED                              [ 26%]
tests/test_graph.py::test_flujo_rag_supervisor_investigador_sintesis PASSED [ 27%]
tests/test_graph.py::test_flujo_con_analista_aporta_los_dos_agentes PASSED [ 29%]
tests/test_graph.py::test_tarea_critica_se_pausa_y_se_reanuda_al_aprobar PASSED [ 31%]
tests/test_graph.py::test_rechazo_humano_no_ejecuta_al_analista PASSED   [ 32%]
tests/test_graph.py::test_tope_de_pasos_corta_el_bucle_del_supervisor PASSED [ 34%]
tests/test_graph.py::test_checkpointer_recuerda_la_conversacion_por_thread_id PASSED [ 36%]
tests/test_graph.py::test_supervisor_recibe_quien_aporto_que PASSED      [ 37%]
tests/test_graph.py::test_validacion_no_deja_cerrar_sin_el_analisis_pedido PASSED [ 39%]
tests/test_graph.py::test_resumen_dice_que_agente_falta PASSED           [ 40%]
tests/test_integracion_redis.py::test_pausa_hitl_se_reanuda_desde_otro_checkpointer SKIPPED [ 42%]
tests/test_integracion_redis.py::test_la_conversacion_sobrevive_a_un_reinicio SKIPPED [ 44%]
SKIPPED [1] tests/test_integracion_redis.py:45: hace falta Redis Stack en redis://localhost:6379 (docker run -p 6379:6379 redis/redis-stack-server)
SKIPPED [1] tests/test_integracion_redis.py:80: hace falta Redis Stack en redis://localhost:6379 (docker run -p 6379:6379 redis/redis-stack-server)
====================== 59 passed, 2 skipped in 7.67s =======================
```

5.4 Lint y despliegue

ruff

```
$ ruff check .
All checks passed!

$ ruff format --check .
43 files already formatted
```

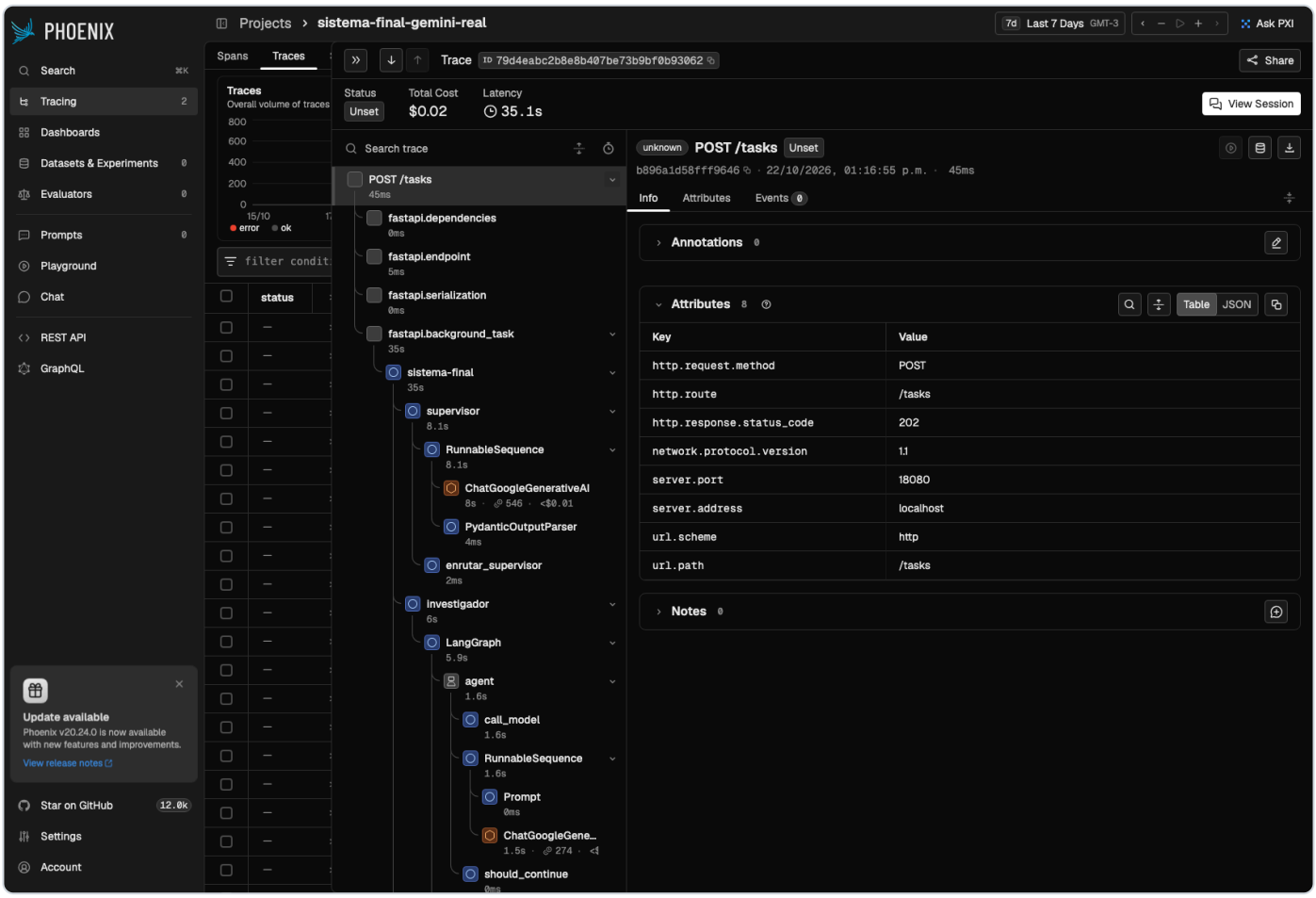
```
docker compose

$ docker compose config --services
redis
ingest
phoenix
app

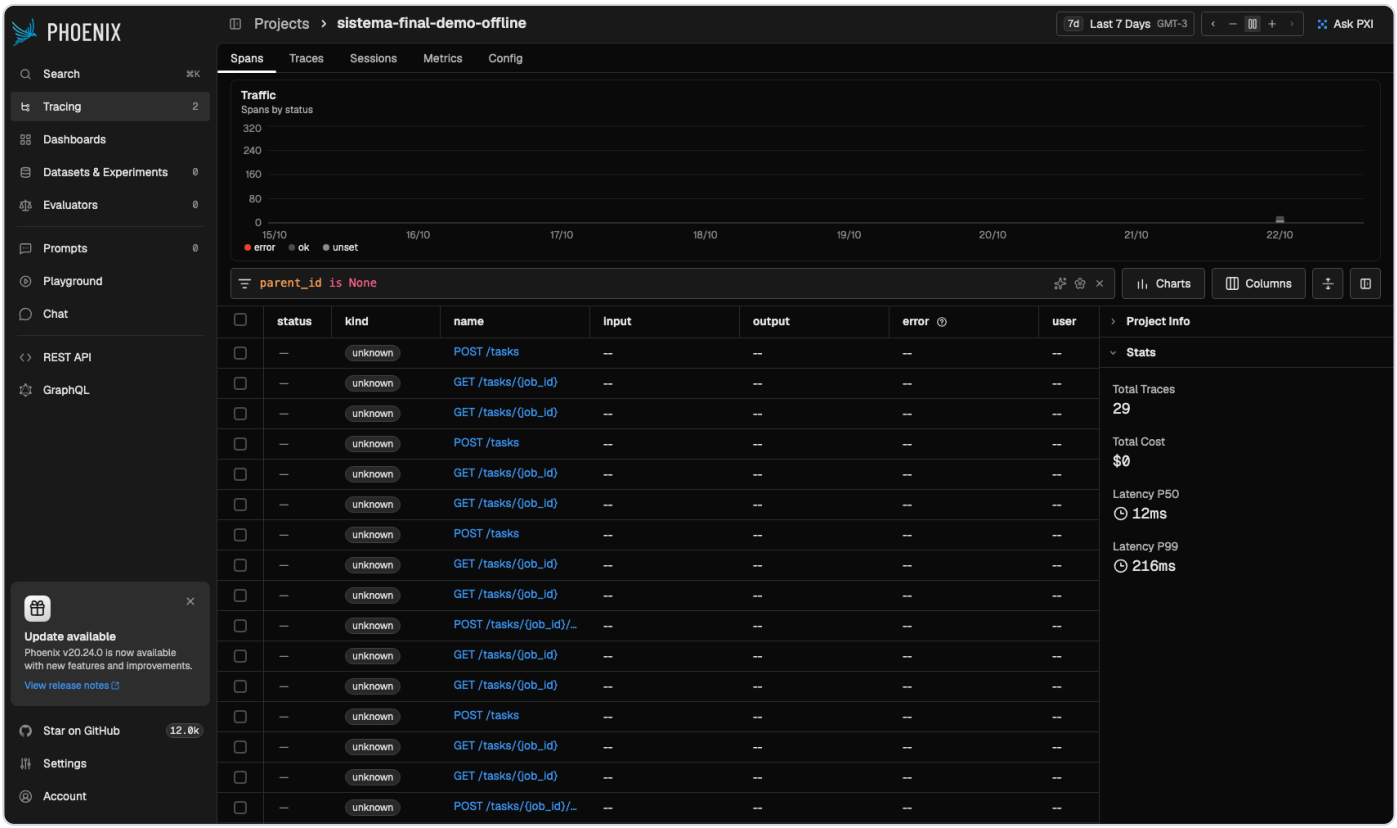
$ docker compose config -q && echo VALIDO
VALIDO
```

6. Trazabilidad (Arize Phoenix)

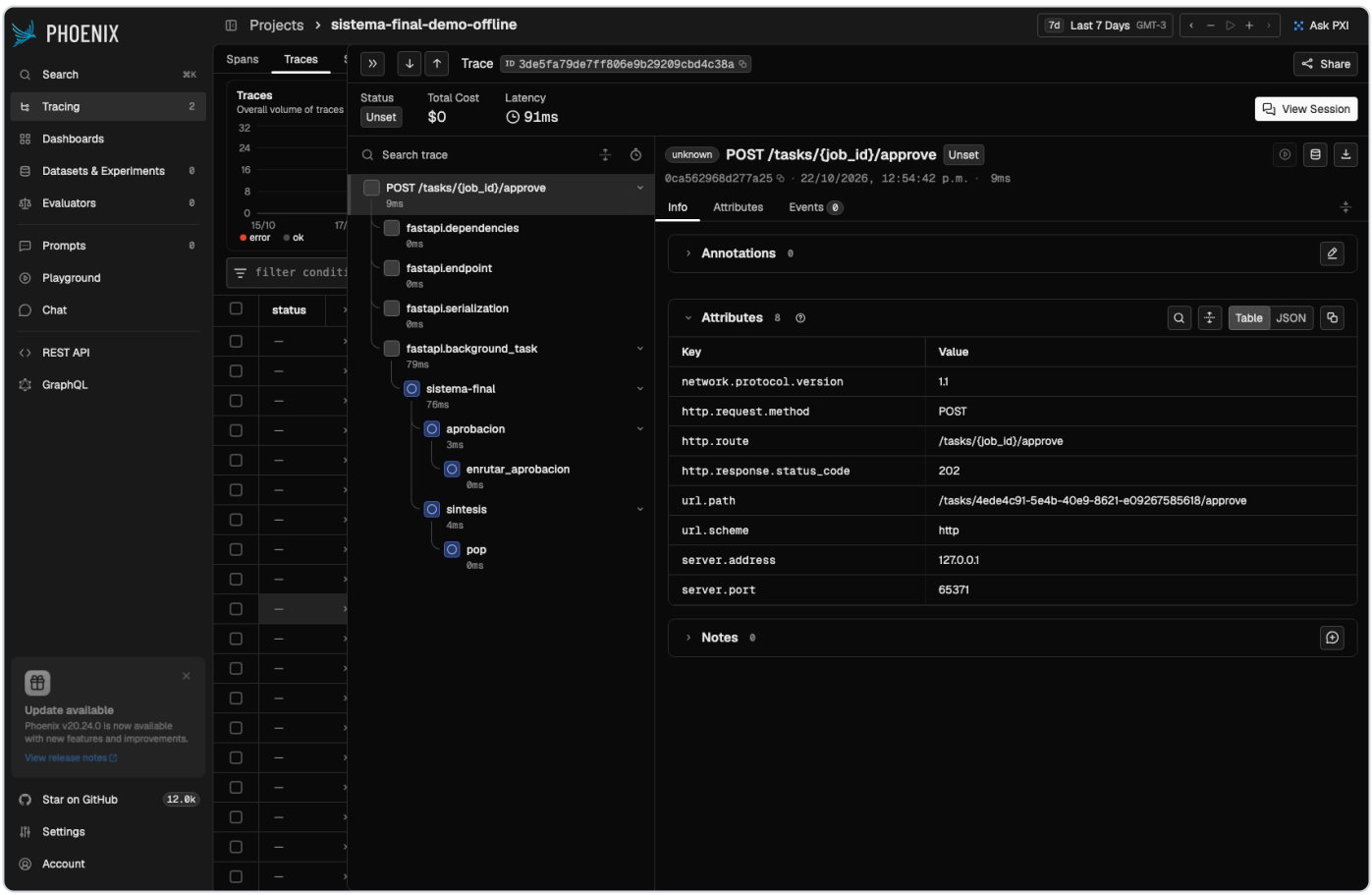
Corrida real con Gemini (`gemini-3.6-flash`): traza de `POST /tasks` con el supervisor, el investigador y la herramienta RAG, con tokens reales y 35 s de latencia. El costo que muestra Phoenix es una estimacion con precio de lista; en el tier gratuito no se cobra.



Las 7 pruebas de `scripts/demo_trazas_offline.py` contra la API real, Redis y Phoenix (LLM con guion, por eso costo $0): 29 trazas, con spans de supervisor, investigador, analista, aprobacion, sintesis, herramientas y del retriever hibrido (`EnsembleRetriever` -> `BM25Retriever` + `VectorStoreRetriever`).



Rechazo humano de una tarea critica (`aprobacion` -> `enrutar_aprobacion` -> `sintesis`):



7. Corrida real con Gemini: las 7 pruebas

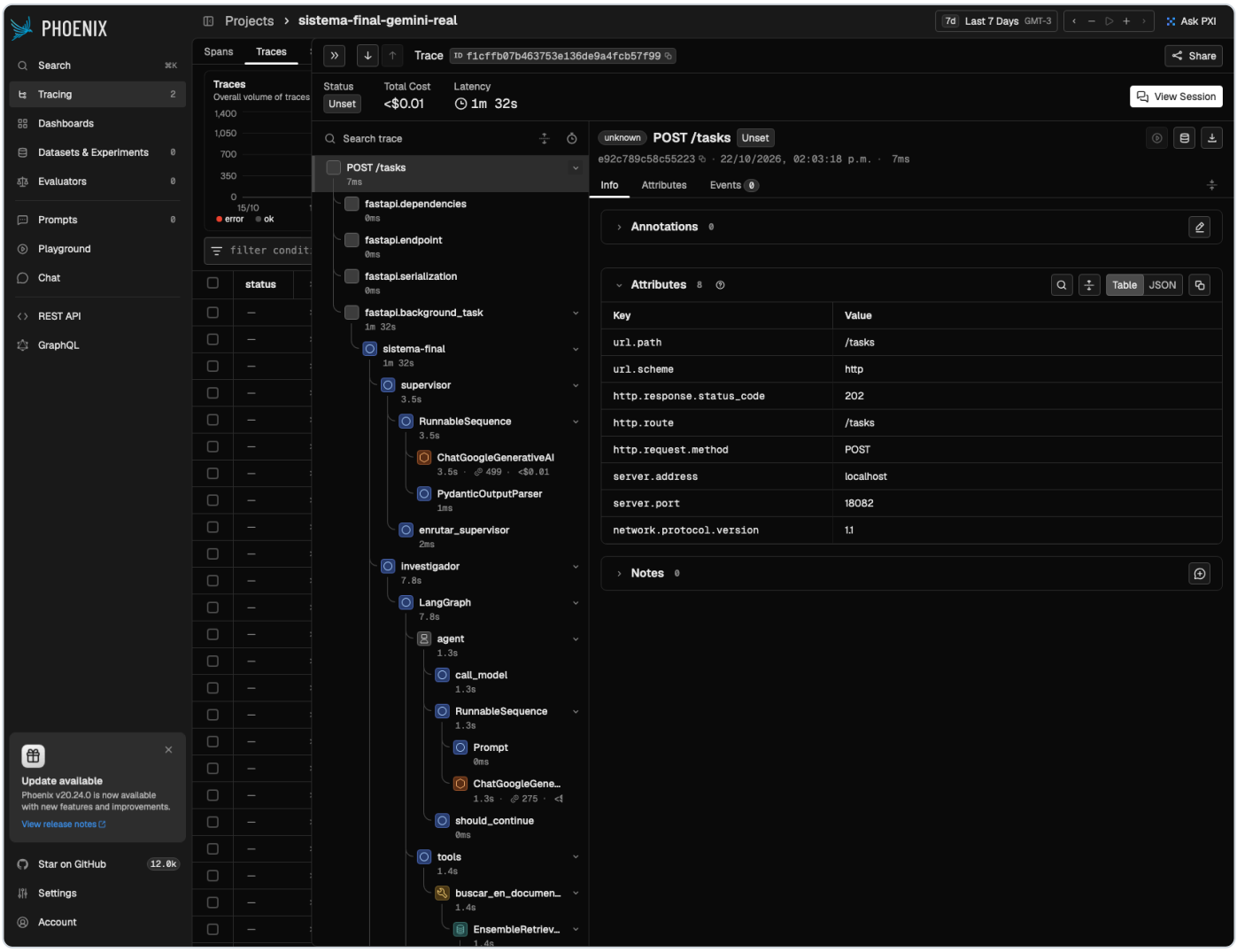
`scripts/pruebas.py` contra la API real (Redis Stack + Phoenix en Docker, `VECTOR_STORE=memory`, embeddings `gemini-embedding-2`). Como el tier gratuito tiene un cupo de 20 pedidos por dia por modelo, las pruebas se repartieron en 3 tandas, cada una con un modelo gratuito distinto. Resultado crudo: `reports/pruebas_gemini_real.json`.

Prueba	Modelo	Estado	Agentes que aportaron	Fuentes citadas	Tiempo
`prueba_1_rag`	`gemini-3.5-flash-lite`	DONE	investigador	rag.md	312.9 s
`prueba_2_rag_y_analisis`	`gemini-3.5-flash-lite`	DONE	investigador, analista	redis.md	205.6 s
`prueba_3_calculo`	`gemini-3.6-flash`	DONE	analista	-	78.4 s
`prueba_4_critica_aprobada`	`gemini-3.6-flash`	DONE (pausa HITL + aprobacion)	investigador, analista	pinecone.md	187.5 s
`prueba_5_critica_rechazada`	`gemini-3.1-flash-lite`	DONE (pausa HITL + rechazo: el analista no corre)	investigador	langgraph.md, redis.md, observabilidad.md	202.4 s
`prueba_6_memoria`	`gemini-3.1-flash-lite`	DONE (2 turnos, mismo `conversation_id`)	investigador	pinecone.md	213.2 s
`prueba_7_validacion`	-	422 (entrada invalida rechazada por Pydantic)	-	-	0 s

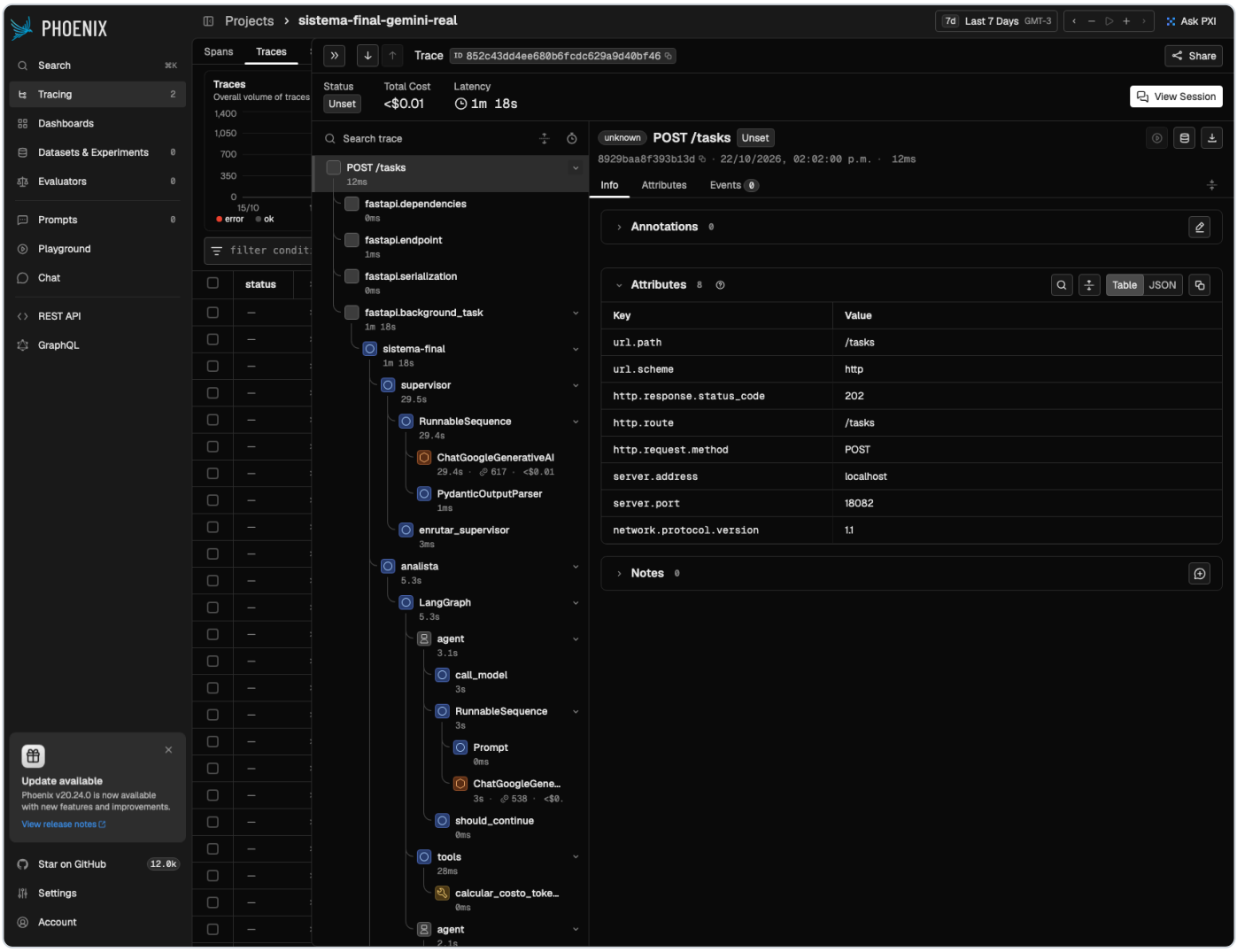
En la prueba 2 actuo la validacion del supervisor con el modelo real: el log registra `validacion: el supervisor quiso cerrar sin el analisis pedido, se deriva al analista`, y despues el analista aporto y la tarea cerro bien.

Phoenix (proyecto `sistema-final-gemini-real`, todas las corridas reales del dia): 54 llamadas al LLM con 39 404 tokens (31 149 de entrada, 8 255 de salida) en 3 modelos, 11 ejecuciones completas del grafo, latencia p50 150 s y p95 268 s. La latencia alta viene de los reintentos por 503/429 del tier gratuito (por ejemplo `gemini-3.6-flash` devolvio 25 errores 503/429 en su tanda), no del sistema.

Pausa humana aprobada (prueba 4), con supervisor, investigador, herramienta RAG y tokens reales:



Analista llamando a `calcular_costo_tokens` (prueba 3):



Detalle de los chequeos de cada componente y de los limites del tier gratuito: `reports/gemini_real.md`.

8. Revision de codigo con OpenCode

Corrida con OpenCode 1.19.4 (agente `plan`, solo lectura) sobre un clone limpio del repo. Salida textual, metadatos y como reproducirla: `REVISION_OPENCODE.md`.



9. Bugs que encontraron los tests y las corridas (todos corregidos)

Bug	Como se detecto	Correccion
BM25 no encontraba terminos pegados a puntuacion ("RediSearch.")	`test_parte_lexica_encuentra_termino_tecnico_exacto` fallo	Tokenizador propio (minusculas, sin acentos ni puntuacion)
Llamadas al LLM terminaban en un mensaje del asistente	Control en todos los tests	Instruccion final como mensaje humano
Rechazar la aprobacion humana igual ejecutaba al analista	`test_rechazo_humano_no_ejecuta_al_analista`	Arista condicional despues de `aprobacion`
Escritura de archivo bloqueante dentro de `async def`	`ruff` (regla ASYNC230)	`asyncio.to_thread`
`gemini-2.5-flash` no disponible para cuentas nuevas (404)	Corrida real	Modelo por defecto configurable, `gemini-3.5-flash-lite`
`gemini-embedding-2` devuelve un embedding agregado si recibe una lista	Documentacion de Google + test	Wrapper propio: un `Content` por texto
El supervisor repetia al investigador	Corrida real con Gemini	El supervisor recibe el resumen de quien aporto que
El supervisor cerraba sin el analisis pedido	Corrida real con Gemini	Validacion en codigo antes de aceptar FINISH
`scripts/pruebas.py` se daba por vencido a los	Corrida real con Gemini (reintentos por 503)	Espera maxima configurable, 900 s por defecto

Bug	Como se detecto	Correccion
120 s mientras el job seguia corriendo		(`PRUEBAS_ESPERA_MAXIMA`)

10. Alcance y limites

- Los tests usan un LLM falso con guion: validan la logica del sistema (enrutamiento, pausa humana, estado, validacion, persistencia, trazas), no la calidad de las respuestas de un modelo.
- La calidad con modelo real se verifico con Gemini (seccion 7): las 7 pruebas pasaron. El tier gratuito tiene un cupo de 20 pedidos por dia y 5 por minuto por modelo (segun el error 429 de Google), por eso se repartieron entre 3 modelos.
- Pinecone no participo en las corridas reales (se uso `VECTOR_STORE=memory`); su integracion esta cubierta con un cliente falso (creacion del indice, chequeo de dimension).